

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

Отчет об исследовательском проекте на тему:
Системы балансировки нагрузки в гибридных распределенных системах

Выполнил студент:

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

Принял руководитель проекта:

Курочкин Илья Ильич
к.т.н., зав.лаб. Ц-1
ИППИ РАН

Москва 2026

Содержание

Аннотация	4
1 Введение	5
1.1 Описание предметной области	5
1.2 Релевантность и значимость	5
1.3 Цели и задачи	6
1.4 Предполагаемые результаты проекта	6
2 Литературный обзор	7
2.1 Обзор предметной области	7
2.2 Обзор существующих методов	8
2.3 Особенности BOINC	9
2.4 Сравнение подходов	9
2.5 Выводы из обзора	10
3 Моделирование системы	11
3.1 Выбор способа моделирования	11
3.2 Топология системы	11
4 Подходы к тестированию системы и метрики качества	13
4.1 Эвристики распределения задач	13
4.2 Метрики оценки производительности	13
4.3 Процесс тестирования	14
5 Разработка алгоритма балансировки	15
5.1 Планировщик BOINC по умолчанию	15
5.2 Требования к кастомному алгоритму	17
5.3 Разработка планировщика задач в гетерогенной системе	17
5.4 Подбор оптимальных параметров для алгоритма	20
5.5 Сопоставление подходов в контексте BOINC	21
5.6 Ограничения разработанного подхода	21
6 Результаты тестирования и сравнение	22
6.1 Стабильная среда	22
6.2 Нестабильная среда: $f_{\text{off}} \approx 32\%$	23

6.3	Нестабильная среда: $f_{\text{off}} = 20\%$	25
6.4	Нестабильная среда: $f_{\text{off}} = 10\%$	26
6.5	Нестабильная среда: $f_{\text{off}} = 50\%$	26
6.6	Нестабильная среда: $f_{\text{off}} \approx 68\%$	27
6.7	Сводное сравнение сценариев	27
6.8	Интерпретация результатов	28
7	Заключение	29
7.1	Достигнутые результаты	29
7.2	Ограничения и компромиссы	29
	Список литературы	31

Аннотация

При проектировании высокопроизводительных и распределенных систем одной из важнейших задач является выбор и настройка системы выдачи заданий (балансировщик нагрузки). При использовании гомогенных вычислительных узлов в кластере можно использовать готовые решения (slurm, torque и др.), но в гетерогенных системах готовые решения не будут работать из коробки. В распределенных системах из-за большой гетерогенности и других ограничений может возникнуть потребность разработки оригинального планировщика заданий. В распределенной вычислительной системе в качестве части ресурсов могут быть использованы облачные ресурсы или ресурсы кластеров. Уже были разработаны решения (к примеру, CluBoRun в грид-системе на платформе BOINC) по использованию ресурсов кластера в фоновом режиме. Предлагается разработать /модифицировать систему выдачи заданий для грид-системы на платформе BOINC, в которой присутствуют сегменты кластеров.

Ключевые слова

Распределенные системы, балансировка нагрузки

1 Введение

1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть
- Кластеры — узлы, интегрированные в распределенную систему с целью повышения доступности
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

1.2 Релевантность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем

- 2 Стандартные планировщики задач оптимизированы под гомогенные системы и не учитывают различия в производительности узлов и их доступность, что в гетерогенных системах приводит к простаиванию некоторых узлов или, наоборот, излишней загруженности отдельных
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Оптимизация всех вычислений, выполняющихся на платформе BOINC

1.3 Цели и задачи

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы
- 5 Анализ полученных результатов

1.4 Предполагаемые результаты проекта

- 1 Подробное описание методики моделирования гибридной распределенной системы
- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Подготовка подробного отчета о проделанной работе и проведенном тестировании

2 Литературный обзор

2.1 Обзор предметной области

В своем начале Интернет имел очень мало потребителей, так что любой сервер мог выдерживать нагрузку даже самого популярного сайта, к тому же его падение не стало бы большой проблемой. Со временем люди были вынуждены заняться вопросами масштабирования ввиду растущей нагрузки. И если сначала проблема решалась увеличением числа процессоров, их улучшением, увеличением объема оперативной памяти, то после стало понятно, что есть потолок такого вертикального масштабирования, так что появилось понятие балансировки нагрузки. [2]

В общем случае, порядок работы балансировщика такой:

- 1 Получение входящих запросов от различных клиентов
- 2 Расчет размера входящих запросов и формирование очереди запросов
- 3 Периодическая проверка состояния серверов в пуле с помощью системы мониторинга
- 4 Использование стратегии / алгоритма / эвристики для выбора подходящего сервера под каждый входящий запрос

Сама идея балансировки нагрузки вносит в систему большое количество преимуществ:

- 1 Упрощенное отслеживания трафика
- 2 Улучшенное использование ресурсов
- 3 Балансировка нагрузки на сеть на основании загруженности серверов
- 4 Улучшенная доступность ресурсов
- 5 Уменьшение избыточного потребления ресурсов

[4]

Решения проблемы балансировки нагрузки делятся на 2 основных подхода в зависимости от того, принимает ли алгоритм решения на основе текущего состояния системы или нет: статический и динамический.

Статический подход предполагает наличие знаний о статусе системы, требованиях к ресурсам, времени связи и другим показателям. Балансировка при таком подходе достигается путем создания связей между списком процессоров и списком задач таким образом, чтобы достигалась максимальная производительность. Хотя назначения могут быть как детерминированными, так

и вероятностными, текущее состояние системы не учитывается ни в одном из вариантов организации балансировки. Это существенный недостаток статических подходов, который оказывает существенное влияние на общую производительность из-за непредсказуемости колебаний нагрузки на систему. Некоторые методы, используемые при статической балансировке: перечисление и поиск в пространстве решений, теория графов, теория очередей.

При динамическом подходе, решения о балансировке нагрузки принимаются на основе текущего состояния системы; задачам разрешается динамически перемещаться с перегруженного узла на недостаточно загруженный узел для более быстрой обработки. Способность реагировать на изменения в системе является основным преимуществом динамического подхода к балансировке нагрузки. Хотя найти динамическое решение намного сложнее, чем статическое, динамическая балансировка нагрузки может повысить производительность за счет своей адаптивности. Далее сосредоточимся именно на таких подходах.

Механизм управления динамической балансировкой может быть распределенным, когда ответственность за балансировку распределяется между узлами, и нераспределенным, когда ответственность за балансировку лежит на одном или нескольких узлах; включает централизованный (один центральный узел управляет балансировкой всей системы) и полу распределенный (узлы делятся на кластеры с центральными узлами для балансировки внутри кластера) подходы. [1]

2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least-Connection) — также просты в мониторинге, но не учитывает общую нагрузку на узлы
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений

- Предиктивные ML подходы — планировщики, которые хорошо работают даже в сложных условиях, но требуют времени и данных для обучения модели [5]
- Существующие решения для BOINC (CluBoRun) — механизмы верификации, учета доступности, отслеживания churn и другие [3]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводит к частому перераспределению задач между узлами. Если в гомогенных системах перенос задач - относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств, поэтому в подобных системах правильным подходом считается исключение или сведение к минимум подобныз операций. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезности для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [4]

2.3 Особенности BOINC

Платформа BOINC использует модель *client-initiated*: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует shared-memory массив feeder, оценивает совместимость workunit с host и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до timeout или повторной репликации. Поле `rsc_frops_est` задаёт оценку сложности, которая может расходиться с реальным временем выполнения - это важный источник ошибок для любой adaptive-политики. Решение CluBoRun [3] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств (кластер / desktop / mobile).

2.4 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочеты:

Таблица 2.1: Сравнение групп подходов к балансировке нагрузки в гибридных системах

Группа методов	Тип	Требования мониторинга	Цель подхода	Минусы
Классические статические	Простые	Низкие	Простое планирование	Не учитывают размер задач
Классические динамические	Реактивные	Низкие/средние	Сократить число одновременных задач на узле	Не учитывает CPU-time узла
Адаптивные методы	Feedback-driven	Средние/высокие	Низкая задержка и оптимальная нагрузка на узлы	Высокие накладные расходы
Предиктивные подходы	ML Обучаемые	Высокие	Минимизация целевой функции ошибки	Требуют данных и обучения
CluBORun	Интеграция кластера / устройств	Низкие/средние	Использование idle ресурсов, учёт доступности	Не решает глобальную балансировку

2.5 Выводы из обзора

- На данный момент существует широкий набор методов балансировки, однако немногие работы систематически исследуют их поведение в гибридных системах
- Основные пробелы: недостаточная оценка влияния churn, упрощённое моделирование сетевых затрат при распределении задач, отсутствие оценки накладных расходов мониторинга и планировщика
- Нет ни одной работы, в которой бы честно рассматривалась балансировка нагрузки в гибридной системе на базе BOINC

Данная работа направлена на устранение выявленных ограничений за счёт оптимизации накладных расходов планировщика, более точного учёта динамической доступности узлов и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

3 Моделирование системы

3.1 Выбор способа моделирования

Для проверки политик балансировки выбран локальный стенд на базе Docker Compose и реального планировщика BOINC 8.3.0, а не discrete-event симулятор или GPU-кластер. Такой выбор обусловлен следующими соображениями:

- Сохраняется полный RPC-цикл BOINC (feeder, feasibility checks, replication, обновление БД)
- Результаты воспроизводимы через скрипты и конфигурационные файлы из артефактов каждого запуска
- Гетерогенность задаётся через Docker cgroup и BOINC preferences без покупки реальной группы устройств
- Чистота эксперимента по CPU-time обеспечивается встроенным в BOINC приложением upercase с параметром `—cpu_time`

Основной эксперимент - CPU-only: отсутствие физических устройств с собственным GPU не позволяет построить корректную модель, варианты с разворачиванием виртуальных машин лишь имитируют наличие GPU. К тому же фокус на GPU размывает основную идею BOINC - volunteer CPU computing.

3.2 Топология системы

Тестовый стенд состоит из 6 хостов-устройств с различными характеристиками:

- 2 кластерных элемента (4 CPU, 4 Gb RAM)
- 1 устройство ПК (2 CPU, 2 Gb RAM)
- 1 маломощный узел (1 CPU, 1 Gb RAM)
- 2 элемента - мобильных устройств (0.5 CPU 0.5 Gb RAM)

Каждое устройство представляет собой независимый вычислительный хост; с точки зрения технической реализации каждый хост - это отдельный Docker контейнер, который конфигурационно настроен под конкретное число CPU. Данный выбор хостов с их настройками был обусловлен следующими требованиями к тестовому стенду:

- Стенд обязан содержать разнородные с точки зрения мощности устройства для моделирования гибридной среды
- Стенд обязан содержать как высокопроизводительные элементы (кластеры), так и низкопроизводительные (мобильные устройства)
- Стенд обязан иметь несколько одинаковых устройств для проверки балансировки нагрузки между однородными хостами (распределение задач между одинаковыми с точки зрения мощности клиентами)

Таким образом и была сформирована модель системы, используемая при тестировании.

Протокол churn Отказы моделируются через `docker compose pause/unpause` на всех шести клиентах с рассинхронизацией старта (`CHURN_STAGGER_SECONDS`). Каждый хост проходит несколько циклов `online → offline → restart`. Доля `offline` в одном цикле:

$$f_{\text{off}} = \frac{T_{\text{offline}}}{T_{\text{online}} + T_{\text{offline}}}.$$

Например, при 25/12 получаем $f_{\text{off}} \approx 32\%$, при 40/10 - 20%, при 12/25 - 68%. Все события также логируются для вычисления метрики Load CV.

Допущения тестового стенда Все 6 хостов физически находятся на одной машине; сеть упрощена; BOINC-клиенты часто сообщают одинаковый `p_frops`, поэтому в код добавлен `host_speed_factor` по `hostname`; `docker pause/unpause` мягче реального `churn` - нет полных отключений; отсутствует миграция задач; `max_wus_to_send=1` и `max_wus_in_progress=2` ограничивают захват очереди задач более быстрыми хостами. Эти допущения не отменяют А/В-сравнение политик на одном стенде, однако не позволяют напрямую экстраполировать абсолютные значения метрик на production BOINC.

4 Подходы к тестированию системы и метрики качества

4.1 Эвристики распределения задач

В литературе помимо классических подходов рассматриваются некоторые эвристики, которые были смоделированы в данном проекте для сравнения с разработанным методом:

- LPT (Longest Processing Time first) - длинные задачи назначаются первыми; при нескольких параллельно работающих однотипных хостах снижает общее время работы, но может ухудшить качество распределения и среднее время отклика для коротких задач
- SJF (Shortest Job First) - короткие задачи получают приоритет; минимизирует среднее время отклика, но крупные задачи могут долго простаивать в очереди

Кластерные планировщики (Slurm, Torque) оптимизированы под гомогенные очереди с централизованным контролем и статическим планированием; они не учитывают pull-модель BOINC и отказы узлов, поэтому не применимы к гибридной системе, по этой причине они не попали в предметную область данного проекта. Таким образом, кроме алгоритма балансировки BOINC по умолчанию были добавлены в тестирование упомянутые выше LPT, SJF, а также классические подходы: Round Robin, Random и Weighted Least Loaded.

4.2 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) – число выполненных задач за единицу времени. Формула: $T = \frac{N_{\text{completed}}}{\Delta t}$
- Время выполнения задачи - для каждой задачи i : $R_i = t_{\text{finish}}^{(i)} - t_{\text{arrival}}^{(i)}$. Среднее: $\bar{R} = \frac{1}{N} \sum_i R_i$
- Загруженность узлов - для узла j : $u_j = \frac{\text{cpu_time}_j}{\text{online_seconds}_j \cdot \text{nrcpus}_j}$. Сводная метрика Load CV $= \sigma(u_j)/\bar{u}$ - коэффициент вариации utilization, а не числа задач; может расходиться с Jain
- Общее время завершения набора задач - $Makespan = \max_i t_{\text{finish}}^{(i)} - \min_i t_{\text{create}}^{(i)}$
- Честность распределения (Jain) - $J = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}$, где x_i - число завершённых задач на хосте i ; в расчёт попадают только хосты с $x_i \geq 1$

- Response - время от создания задачи до получения результата сервером; p95 - 95-й перцентиль

Такой набор метрик позволяет оценить и скорость системы (throughput, makespan, response), и качество распределения (Jain, Load CV). Отдельно анализируется согласованность типов задач с классами устройств (таблицы short/medium/long по хостам).

4.3 Процесс тестирования

Система тестировалась в шести сценариях режима: одна стабильная среда и пять вариантов churn с разными $(T_{\text{online}}, T_{\text{offline}})$. Метрики автоматически вычисляются скриптом `analyze_metrics.py` и сохраняются в файл `analysis/metrics.json`; параметры прогона - в файл `manifest.json`.

В качестве источника задач использовалось приложение `upercase` с параметром `-cpu_time`. Нагрузка: 36 задач - 18 short (5 s), 12 medium (30 s), 6 long (120 s). Основная цель кастомного алгоритма - выровнять число задач на хостах (Jain) и направить long/medium на мощные узлы, а short - на phone/low-power, найдя компромисс с остальными метриками.

Таблица 4.1: Сценарии тестирования всех политик за раз

Churn	Online (s)	Offline (s)	f_{off}
stable	—	—	0
heavy	45	5	10%
heavy	40	10	20%
heavy	25	12	32%
heavy	30	30	50%
heavy	12	25	68%

Матрица экспериментов

5 Разработка алгоритма балансировки

Прежде чем описывать разработанный алгоритм, необходимо зафиксировать, как именно работает штатный планировщик в BOINC, поскольку именно он выступает baseline в экспериментах и служит границей, ниже которой по метрикам опускаться неприемлемо.

5.1 Планировщик BOINC по умолчанию

BOINC использует pull-модель распределения задач: вычислительный узел самостоятельно инициирует RPC-запрос к планировщику и сообщает, сколько работы он готов принять (в секундах CPU, в числе задач или иначе). Планировщик не выбирает хост для задачи - он лишь отвечает на запрос данного клиента в данный момент. Назначенная задача не мигрирует на другой узел по достижении таймаута, при ошибке или при создании новой реплики. Это принципиально отличает BOINC от централизованных кластерных планировщиков (Slurm, Torque), где планировщик видит всю очередь задач и все узлы одновременно, и именно эти недостатки должны быть устранены или сведены к минимуму в рамках текущего проекта.

Очередь задач, доступная планировщику, - это не вся база данных, а кэш в памяти, который заполняет отдельный процесс под названием *feeder*. Он читает из БД информацию о задачах, готовых к отправке, и помещает их в массив фиксированного размера (по умолчанию до 100 слотов, параметр *shmem_work_items* задаётся конфигурационно). Каждый элемент содержит метаданные задачи (оценку сложности *rsc_flops_est*, требования к памяти и диску), приоритет реплики *res_priority* и счётчик отказов *infeasible_count*. Планировщик при каждом запросе клиента сканирует этот массив; порядок обхода начинается со случайного смещения, чтобы не застревать на одних и тех же задачах.

Для вычисления наиболее подходящего хоста вызывается функция *send_work_score()*, логика которой сводится к двум шагам:

- 1 Сканирование и ранжирование - для каждого слота в shared memory подбирается совместимая версия приложения под платформу хоста, для каждой допустимой пары (*workunit*, *host*) вычисляется *score*, после чего все кандидаты сортируются по *score* по убыванию
- 2 Отправка - планировщик последовательно пытается выдать задачи из отсортированного списка, пока не исчерпан лимит RPC *max_wus_to_send*, не превышены квоты *max_jobs_in_progress* и пока хост ещё запрашивает работу

Отдельный проход выполняется для каждого типа процессора (GPU-типы, затем CPU).

Прежде чем хост попадёт в ранжирование, он должен пройти дополнительные проверки:

- Достаточно ли оперативной памяти и дискового пространства?
- Приложение входит в список, выбранный пользователем (хостом) в project preferences?
- Хост успеет завершить задачу до глобального дедлайна по ней?

Последняя проверка - ключевая для гетерогенной среды. Оценка длительности выполнения задачи:

$$\hat{t}_{ij} = \frac{\text{rsc_flops_est}_i}{\text{projected_flops}_j \cdot \text{available_frac}_j},$$

где available_frac - доля времени, когда клиент реально доступен для BOINC. К этой оценке добавляется estimated_delay - время, которое хост уже занят текущими задачами. Если $\hat{t}_{ij} + \text{estimated_delay} > \text{delay_bound}$, задача для данного хоста в score не попадает. Следствие: мощные и стабильные узлы забирают длинные задачи; слабые и малодоступные - только короткие или ничего.

Если быстрые проверки пройдены, выполняется slow_check() с запросами в БД: нет ли уже результата этой задачи на данном хосте. Только после этого задача добавляется в ответ клиенту.

Оценка каждого хоста считается исходя из нескольких эвристик:

- Reliability - задача с need_reliable=true и высоким приоритетом отправляется только на хосты с подходящим уровнем приложения
- LTT - для high-priority workunits в режиме batch acceleration задачи идут только на LTT-хосты
- Locality - бонус, если входные данные задачи уже есть на хосте
- res_priority - приоритет реплики из БД добавляется напрямую к оценке
- App preferences - учёт пожеланий пользователя

Итоговый score - сумма этих слагаемых.

Что baseline оптимизирует и чего не делает. Штатный планировщик оптимизирует локальную выгоду для текущего RPC с жёстким требованием уложиться в дедлайн задачи. Он не стремится к равномерному распределению задач между хостами, не учитывает глобальную загрузку хостов, не различает классы устройств - только доступность конкретного хоста и его характеристики, не сопоставляя хосты между собой и с типами задач. Таким образом, возникает проблема: мощные хосты забирают любые задачи, которые им предлагает планировщик, а более слабым узлам достаются не самые простые с точки зрения требуемой вычислительной мощности задачи.

5.2 Требования к кастомному алгоритму

Исходя из обзора литературы, ограничений штатного планировщика BOINC и целей данной работы, к разрабатываемому алгоритму были сформулированы следующие требования:

- 1 Минимальная инвазивность - сохранить весь контракт штатного scheduler: feeder, feasibility checks, replication, locking shared memory, обновление БД. Меняется только функция ранжирования допустимых кандидатов
- 2 Size-aware dispatch - ключевая идея: точное сопоставление сложности задачи с производительностью узла для более честного распределения задач
- 3 Churn-awareness - учёт динамической доступности узла, чтобы не отправлять тяжёлые задачи на малодоступные хосты
- 4 Отсутствие миграции - не вводить перенос задач между хостами, поскольку это несовместимо с моделью BOINC и неоправданно дорого в гетерогенной нестабильной среде
- 5 Минимизация накладных расходов - решение принимается за один проход без глобального мониторинга и без обращений к БД сверх штатных проверок
- 6 Сравнимость - помимо целевой политики реализовать контрольные алгоритмы (random, LPT, SJF, Round Robin, Weighted Least Loaded) для сравнения качества распределения и накладных расходов

5.3 Разработка планировщика задач в гетерогенной системе

Модификация выполнена в модуле sched_send.cpp. При включенном feature flag - custom_load_bal - вместо send_work_score() вызывается send_work_custom(). Функция send_work_custom() повторяет структуру send_work_score():

- Сканирование ssp->wu_results[] со случайным offset
- Подбор app_version и feasibility через те же get_score(), wu_is_infeasible_fast(), slow_check()
- Сортировка кандидатов по score и отправка top-N

Отличие - в методе custom_score(), которая заменяет штатный JOB::get_score() для вычисления итоговой оценки хоста.

Поскольку Docker-клиенты часто сообщают одинаковый `p_frops`, для `hybrid` введён коэффициент `host_speed_factor = v_j` по числу CPU узла. Спрогнозированное время в этом случае: $\hat{t}_{ij} = \text{estimate_duration}(i, j)/v_j$.

Основной идеей было взять за основу оценку `baseline`-алгоритма и дополнить ее членами, явно учитывающими размер задачи, доступность хоста и "соответствие" мощности хоста размеру задачи. Целевая политика - `hybrid`, остальные служат контрольными для изоляции отдельных гипотез.

Контрольные политики Все контрольные политики используют защищённый `baseline score` ($100 \times s^{\text{BOINC}}$), чтобы штатные факторы (`reliability`, `locality`) сохраняли влияние, а экспериментальный член мог их сдвинуть:

Таблица 5.1: Реализованные политики `dispatch`

Политика	Суть ranking
<code>baseline</code>	штатный <code>send_work_score()</code>
<code>random</code>	<code>baseline</code> + детерминированный псевдослучайный шум от <code>result_id</code> и <code>host_id</code>
<code>lpt</code>	<code>baseline</code> + $\log(1 + \hat{t}_{ij})$ - длинные задачи первыми
<code>sjf</code>	<code>baseline</code> - $\log(1 + \hat{t}_{ij})$ - короткие задачи первыми
<code>round_robin</code>	<code>baseline</code> + циклический бонус по <code>result_id mod 6</code> и <code>host_id</code>
<code>weighted_least_workload</code>	<code>baseline</code> + $\log(1 + v_j \cdot a_j / (L_j + 1))$, где L_j - число задач in-progress на хосте
<code>hybrid</code>	кастомный score

Здесь v_j - `host_speed_factor` (число CPU у хоста), a_j - `cpu_available_frac`.

Кастомная политика Для пары (`workunit i`, `host j`) вычисляется составной score. Сначала определяются вспомогательные величины:

$$a_j = \text{clamp}(\text{cpu_available_frac}_j, 0.15, 1.0),$$

$$\tilde{t}_{ij} = \hat{t}_{ij}/a_j,$$

$$\rho_{ij} = \text{clamp}(\tilde{t}_{ij}/T_{\text{target}}, 0.01, 100),$$

где $T_{\text{target}} = 30$ секунд - целевое время выполнения (параметр `custom_lb_target_runtime`, близко к длительности `medium`-задач). Далее:

$$\begin{aligned} \text{size_affinity}_{ij} &= -|\log \rho_{ij}| \cdot a_j && \text{штраф за несоответствие размера задачи целевому} \\ \text{deadline_press}_i &= T_{\text{target}} / \text{delay_bound}_i && \text{бонус срочным workunit,} \\ \text{runtime_penalty}_{ij} &= \log(1 + \tilde{t}_{ij} / T_{\text{target}}) && \text{штраф за длинные задачи на текущем хосте,} \\ \text{host_fit}_{ij} &= (\log(1 + v_j) - \log(1 + \tilde{t}_{ij})) \cdot a_j^2 && \text{бонус за соответствие мощности хоста длительности} \end{aligned}$$

Дополнительно вводится `slow_host_long_penalty` - усиленный штраф за отправку `medium/long` задач на маломощные узлы, масштабируемый относительно T_{target} и v_j .

Baseline score масштабируется с учётом доступности хоста, что ослабляет влияние `reliability` / `locality` на нестабильных хостах и усиливает на стабильных - важно для сценариев с отказами узлов.

$$s_{ij}^{\text{base}} = (25 + 55 \cdot a_j) \cdot s_{ij}^{\text{BOINC}},$$

Итоговый score выглядит так:

$$\begin{aligned} \text{score}_{ij} &= s_{ij}^{\text{base}} + w_{\text{size}} \cdot \text{size_affinity}_{ij} + w_{\text{dl}} \cdot \text{deadline_press}_i \\ &\quad - w_{\text{rt}} \cdot \text{runtime_penalty}_{ij} + 0.05 \cdot w_{\text{size}} \cdot \text{host_fit}_{ij} - w_{\text{rt}} \cdot \text{slow_host_long_penalty}_{ij}, \end{aligned}$$

Все веса задаются в `config.xml`: $w_{\text{size}} = 2$, $w_{\text{dl}} = 5$, $w_{\text{rt}} = 0.55$.

Интерпретация элементов score

- `size_affinity` наказывает задачи, чьё спрогнозированное время работы сильно отличается от T_{target} , сильнее на малодоступных хостах - стимулирует более честное назначение задачи в текущем контексте
- `host_fit` поощряет отправку задач на хосты, чья мощность v_j согласована с $\tilde{t}_{ij}$: мощный хост + длинная задача или слабый + короткая
- `runtime_penalty` и `slow_host_long_penalty` не дают накапливать длинные задачи на маломощные узлы
- `deadline_press` предотвращает голодание задач с коротким таймаутом
- Множитель $(10 + 10 \cdot a_j)$ вместо 100 у контрольных политик - компромисс: baseline-факторы

BOINC сохраняют значимость, но кастомные слагаемые также могут играть важную роль при формировании финальной оценки хоста

Параметры конфигурации Переключение политик и весов выполняется через файл `config.xml` без пересборки:

- `custom_load_balancer` - 0 (baseline) / 1 (custom)
- `custom_lb_policy` - имя политики
- `custom_lb_target_runtime`, `custom_lb_size_weight`, `custom_lb_deadline_weight`, `custom_lb_runtime_w`
- `max_wus_to_send=1`, `max_wus_in_progress=2` - ограничения стенда, исключающие захват всей очереди одним быстрым клиентом

5.4 Подбор оптимальных параметров для алгоритма

Веса кастомного алгоритма ($w_{size} = 2$, $w_{dl} = 5$, $w_{rt} = 0.55$, $T_{target} = 30$ s) подбирались итеративно по серии прогонов baseline vs hybrid в нестабильной среде с отказами до получения устойчивого преимущества hybrid по Jain при приемлемой потере в общей продолжительности работы и пропускной способности. Ключевые решения:

- снижение множителя baseline с 100 до $(10 + 10 \cdot a_j)$ - кастомные слагаемые получают влияние на оценку, но уже посчитанный внутри BOINC score не обнуляется, поскольку цель проекта модифицировать текущую систему, а не написать с нуля. Увеличение коэффициента приводит к соишком высокой корреляции между обеими политиками; при слишком низком - слагаемое теряется на фоне других. Оптимальный коэффициент получился от 15 до 30, после вычисления которого была добавлена привязка к доступности - решение для более качественного учета churn
- $T_{target} = 30$ s - сопоставимо с размером medium-задач. Увеличение или уменьшение приводит к значительному нарушению равномерности распределения задач между хостами и заметно ухудшает Jain
- $w_{rt} = 0.55$ - баланс между штрафом long на слабых хостах и допустимым makespan

Чувствительность: увеличение w_{size} или w_{rt} на 30-50% повышает Jain, но увеличивает makespan hybrid на 5-15%; уменьшение возвращает поведение к baseline. Текущая точка - компромисс по результатам прогонов политик в разных средах.

5.5 Сопоставление подходов в контексте BOINC

Таблица 5.2: Классические методы и их реализация на pull-модели BOINC

Метод	Типичная push-LB	Реализация в проекте	Ограничение в BOINC
Round Robin	глобальная очередь	round_robin	только local RPC
Least Loaded	мониторинг всех узлов	weighted_least_loaded	только in-progress текущего хоста
LPT / SJF	централизованный порядок	lpt, sjf	ranking в feeder, без migration
Random	равновероятный выбор	random	детерминированный hash
Hybrid (наш)	multi-criteria	hybrid	size + availability + host_fit

5.6 Ограничения разработанного подхода

- Локальность решений - планировщик не видит глобального распределения задач между однородными хостами: они имеют схожий baseline score, поэтому возможна неравномерность распределения похожих задач между такими хостами, что на самом деле не так и важно при сохранении схожей нагрузки на однородных узлах
- Зависимость от оценок - $\hat{t}_{ij}$ строится на rsc_frops_est, которое может расходиться с реальным CPU-time
- Компромисс fairness vs makespan - усиление host_fit и штрафов за long на слабых узлах улучшает равномерность и распределение по типам устройств, но увеличивает суммарное время работы относительно других политик, которые "сливают" мелкие задачи на кластеры и минимизируют продолжительность работы
- Нет миграции при churn - если хост перестал быть доступен с задачей in-progress, та ждёт таймаута и репликации; кастомный алгоритм лишь снижает вероятность отправки тяжёлых задач на нестабильные хосты до их назначения

6 Результаты тестирования и сравнение

Проведено шесть полных прогонов по всем политикам. Ниже - метрики по каждому сценарию и сводное сравнение. В таблицах используются следующие столбцы:

- Policy - текущая политика
- Throughput - пропускная способность (задач в секунду)
- Response - среднее время ожидания взятия задачи в работу (в секундах)
- Response p95 - 95-й перцентиль response
- Makespan - общее время выполнения набора задач (в секундах)
- Load CV - ковариация нагрузки на узлы
- Jain - равномерность распределения задач

Ниже эксперименты и короткие выводы по ним:

6.1 Стабильная среда

Таблица 6.1: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.117	124.2	187.5	252	0.368	0.725
round_robin	0.116	117.7	197.5	266	0.449	0.812
weighted_least_loaded	0.109	117.6	206.5	274	0.711	0.761
sjf	0.109	119.2	214.0	272	0.453	0.818
random	0.111	156.9	220.0	269	0.350	0.628
lpt	0.095	144.8	190.3	324	0.451	0.688
hybrid	0.096	140.4	217.8	338	0.758	0.923

Как видим, кастомный метод проиграл планировщику по умолчанию (да и остальным политикам) по всем временным метрикам: продолжительность работы (338 секунд, на 30% больше, чем у baseline), пропускная способность (в среднем на 10-15% хуже остальных), время простоя задач (в p95 хуже него только random). Классические методы Round Robin и Weighted Least Loaded приближены по качеству к baseline, однако тот показывает лучшую ковариацию нагрузки (0.368).

Эвристики также заметно проигрывают по скорости работы планировщику ”из коробки”. Единственная метрика, по которой самописный планировщик опережает другие политики - это равномерность распределения задач по хостам - 0.923, что на 15-30% лучше остальных алгоритмов балансировки. Ниже представлена статистика по хостам:

Таблица 6.2: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	12	10	5	3	4	2	36
round_robin	10	10	5	4	3	4	36
weighted_least_loaded	9	12	5	4	3	3	36
sjf	9	10	7	3	3	4	36
random	14	10	6	2	2	2	36
lpt	11	12	5	2	4	2	36
hybrid	8	8	6	3	6	5	36

По данной таблице становится очевидным преимущество написанного алгоритма - за счет своей архитектуры он не позволяет кластерам забирать большую часть задач для исполнения, а равномерно распределяет их по всем хостам (суммарно на оба кластера приходится 44% задач против 61% у планировщика по умолчанию). Качество такого распределения еще подробнее можно увидеть на таблице ниже со статистикой разбора задач каждым хостом по типам:

Таблица 6.3: Число завершённых задач по типам и хостам

	Host						Host				
		short	medium	long	Σ			short	medium	long	Σ
baseline	cluster-01	7	4	1	12	hybrid	cluster-01	3	4	1	8
	cluster-02	5	2	3	10		cluster-02	1	2	5	8
	desktop-01	2	2	1	5		desktop-01	2	4	0	6
	low-power-01	1	2	0	3		low-power-01	1	2	0	3
	phone-01	3	1	0	4		phone-01	6	0	0	6
	phone-02	0	1	1	2		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

Здесь преимущества становятся более очевидными: планировщик по умолчанию отдает почти все short-задачи (66%) мощным кластерам, так что слабые устройства вынуждены разбирать средние и тяжелые задачи, тогда как в случае с кастомным алгоритмом кластеры забирают все long-задачи и половину medium, оставляя легкие маломощным узлам.

6.2 Нестабильная среда: $f_{\text{off}} \approx 32\%$

Churn: online 25 s, offline 12 s.

Таблица 6.4: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.121	119.4	198.8	243	0.329	0.806
round_robin	0.112	120.2	209.3	277	0.465	0.857
weighted_least_loaded	0.103	133.9	229.0	295	0.459	0.893
sjf	0.110	129.4	223.5	290	0.587	0.851
random	0.085	161.5	213.3	363	0.399	0.610
lpt	0.122	141.4	196.8	237	0.344	0.617
hybrid	0.090	146.8	229.5	344	0.654	0.871

Таблица 6.5: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	11	9	5	4	4	3	36
round_robin	9	9	7	4	4	3	36
weighted_least_loaded	8	9	7	4	4	4	36
sjf	9	10	5	4	4	4	36
lpt	12	13	5	2	2	2	36
hybrid	8	10	5	3	5	5	36
random	9	15	6	2	2	2	36

Таблица 6.6: Число завершённых задач по типам и хостам

baseline	Host	short	medium	long	Σ	hybrid	Host	short	medium	long	Σ
	cluster-01	7	4	1	11		cluster-01	3	0	5	8
	cluster-02	4	2	3	9		cluster-02	3	6	1	10
	desktop-01	2	2	1	5		desktop-01	1	4	0	5
	low-power-01	3	1	0	4		low-power-01	1	2	0	3
	phone-01	2	1	1	4		phone-01	5	0	0	5
	phone-02	0	3	0	3		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

Видно, что все преимущества самописного алгоритма в стабильной среде успешно перенесли на нестабильную с постоянными отказами: качество распределения задач (0.871) так же лучше чем у любой другой политики; кластеры так же забирают на себя все сложные задачи, оставляя легкие для более слабых узлов; он все еще проигрывает по времени (makespan 344 секунды, что значительно больше, чем у остальных), но потери в ключевых метриках составляют около 10-15%, что является по своей сути компромиссом для лучшего распределения задач.

6.3 Нестабильная среда: $f_{\text{off}} = 20\%$

Churn: online 40 s, offline 10 s.

Таблица 6.7: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.119	134.9	186.3	255	0.477	0.844
round_robin	0.102	127.9	217.5	285	0.591	0.850
weighted_least_loaded	0.111	127.0	221.0	276	0.519	0.857
sjf	0.103	128.8	220.5	294	0.608	0.857
random	0.114	145.3	216.3	262	0.450	0.610
lpt	0.113	150.3	213.8	264	0.320	0.610
hybrid	0.097	142.7	214.5	335	0.758	0.871

Таблица 6.8: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	10	9	5	3	5	4	36
round_robin	9	10	5	4	4	4	36
weighted_least_loaded	9	9	7	4	4	3	36
sjf	9	9	7	4	3	4	36
random	14	11	5	2	2	2	36
lpt	14	11	5	2	2	2	36
hybrid	8	10	5	3	5	5	36

Таблица 6.9: Число завершённых задач по типам и хостам

baseline	Host	short	medium	long	Σ	hybrid	Host	short	medium	long	Σ
	cluster-01	6	2	2	10		cluster-01	0	3	5	8
	cluster-02	4	2	3	9		cluster-02	5	4	1	10
	desktop-01	1	3	1	5		desktop-01	2	3	0	5
	low-power-01	0	3	0	3		low-power-01	1	2	0	3
	phone-01	4	1	0	5		phone-01	5	0	0	5
	phone-02	3	1	0	4		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

В этом эксперименте baseline продолжает лидировать по makespan (255 секунд) и пропускной способности (0.119), а кастомный алгоритм сохраняет лучшее качество распределения (0.871) и снова проигрывает по makespan на 30%. По сравнению с прошлым экспериментом самописный алгоритм теряет меньше по пропускной способности (-19% против -26% ранее).

Отметим, что по таблице может сложиться мнение о некачественном распределении long между реер-кластерами (например, 5:1). Это следствие локальности pull-модели: планировщик не видит глобальную загрузку cluster-01/02. При этом phone/low-power не получают long, а Jain остаётся лучшим среди политик.

6.4 Нестабильная среда: $f_{\text{off}} = 10\%$

Churn: online 44 s, offline 5 s.

Таблица 6.10: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
hybrid	0.126	135.2	195.8	226	0.597	0.923
round_robin	0.121	121.1	190.3	252	0.561	0.878
random	0.121	144.9	197.8	254	0.432	0.639
lpt	0.115	152.1	210.0	258	0.318	0.643
sjf	0.118	115.6	215.3	265	0.468	0.837
weighted_least_loaded	0.113	119.3	210.0	270	0.622	0.761
baseline	0.094	137.6	200.3	350	0.679	0.831

В этом сценарии hybrid впервые лидирует по makespan (226 s) и throughput (0.126), сохраняя лучший Jain (0.923). Baseline проседает (makespan 350 s) - при редких, но частых RPC churn availability-aware ranking перераспределяет long с нестабильных узлов эффективнее, чем чистый deadline-score.

6.5 Нестабильная среда: $f_{\text{off}} = 50\%$

Churn: online 30 s, offline 30 s.

Таблица 6.11: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
random	0.109	171.5	222.5	289	0.474	0.639
baseline	0.104	153.1	221.0	287	0.494	0.806
lpt	0.106	165.0	233.0	294	0.377	0.587
round_robin	0.097	140.7	226.8	320	0.344	0.931
sjf	0.102	145.0	230.3	304	0.279	0.871
hybrid	0.093	157.5	234.3	339	0.622	0.900
weighted_least_loaded	0.085	145.9	255.5	370	0.590	0.878

При 50% offline round_robin достигает Jain=0.931 (лучший в сценарии), baseline - лучший makespan (287 s). Hybrid сохраняет Jain $\geq$ 0.90, но проигрывает по времени - при длительных offline size-aware политика консервативно отдаёт short на edge-узлы, увеличивая makespan.

6.6 Нестабильная среда: $f_{\text{off}} \approx 68\%$

Churn: online 12 s, offline 25 s.

Таблица 6.12: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
lpt	0.115	186.6	240.0	260	0.346	0.610
round_robin	0.100	148.6	269.8	305	0.456	0.885
baseline	0.100	192.2	273.3	306	0.498	0.812
sjf	0.104	149.3	249.3	302	0.517	0.893
weighted_least_loaded	0.099	151.9	265.5	317	0.485	0.931
random	0.088	179.3	250.0	360	0.626	0.735
hybrid	0.084	179.9	289.0	377	0.624	0.915

При экстремальном churn лидирует lpt по makespan (260 s); hybrid сохраняет Jain=0.915, но makespan 377 s - availability-aware штрафы и size-matching не компенсируют частые длинные простои. Weighted least loaded достигает Jain=0.931, baseline - средний баланс.

6.7 Сводное сравнение сценариев

Таблица 6.13: Лидеры по makespan и Jain в каждом прогоне

f_{off}	Best makespan	Makespan (s)	Best Jain	Jain	Hybrid: Jain / Makespan
Stable FULL	baseline	252	hybrid	0.923	0.923 / 338
32	lpt	237	wleast	0.893	0.871 / 344
20	baseline	255	hybrid	0.871	0.871 / 335
10	hybrid	226	hybrid	0.923	0.923 / 226
50	baseline	287	round_robin	0.931	0.900 / 339
68	lpt	260	wleast	0.931	0.915 / 377

Таблица 6.14: Throughput (tasks/s) hybrid и baseline по сценариям

Policy	Stable	32	20	10	50	68
baseline	0.117	0.121	0.119	0.094	0.104	0.100
hybrid	0.096	0.090	0.097	0.126	0.093	0.084

6.8 Интерпретация результатов

- Baseline оптимален по makespan/throughput в stable и умеренном churn (40/10, 30/30), минимизируя общее время через концентрацию short на кластерах
- Разработанный алгоритм стабильно лидирует по Jain (0.87-0.92) во всех сценариях; единственный случай победы по makespan - средняя доступность в 10%, где короткие offline-окна и availability-aware ranking совпадают с целями алгоритма
- LPT выигрывает по makespan, но Jain низкий (0.61-0.62) - long концентрируются на cluster
- Round Robin / Weighted Least Loaded - компромисс: близки к baseline по времени, Jain выше из-за особенностей алгоритмов (распределять задачи по хостам примерно поровну)
- Random был добавлен скорее как второй baseline, чтобы разработанный алгоритм не оказался вдруг хуже случайного, и он очень нестабилен от запуска к запуску, что было ожидаемо
- Наблюдается устойчивый trade-off: +15-30% у времени работы hybrid относительно baseline при выигрыше в 15-27% Jain и корректном распределении short/long задач по классам устройств

7 Заключение

В рамках проекта был разработан и экспериментально проверен кастомный балансирующий задач для гетерогенной BOINC-среды. Модификация интегрирована в планировщик через feature flag и меняет только оценку допустимых задач, сохраняя имеющуюся структуру: feeder, feasibility checks и replication.

7.1 Достигнутые результаты

- 1 Реализованы семь политик планирования задач: baseline, random, lpt, sjf, round_robin, weighted_leas, hybrid
- 2 Построен Docker-стенд из 6 гетерогенных хостов с контролируемы отказами; проведены 6 прогонов по всем политикам со сбором метрик
- 3 Разработанный алгоритм стабильно обеспечивает наилучшую равномерность (0.87-0.92) и size-aware распределение: большие задачи на мощные узлы, а мелкие на более слабые - чего не обеспечивал никакой другой алгоритм
- 4 В сценарии с кратковременными отказами (доступность на уровне 90%) кастомный алгоритм одновременно лидирует по общей продолжительности работы (226 s), пропускной способности (0.126) и равномерности распределения (0.923) - доказывая, что алгоритм применим не только в нестабильной среде, но и в приближенной к стабильной, где у штатного планировщика он выигрывает уже по большинству метрик

7.2 Ограничения и компромиссы

- В большинстве сред политики baseline/LPT быстрее на 10-30% по общему времени разбора задач в сравнении с разработанным алгоритмом - сознательно жертвуем скоростью ради равномерности и честности распределения
- Pull-модель BOINC не даёт глобального баланса между однородными кластерами (при всех прогонах распределение наиболее тяжелых задач 1 к 5) - баланс достигается комплексно за счет грамотного распределения всех типов задач
- Оценки rsc_frops_est и локальный учет активных задач на хосте ограничивают точность алгоритма в реальной среде

В общем и целом цель работы - разработка системы выдачи заданий для гибридной BOINC-среды - достигнута: алгоритм разработан и реализован, протестирован в различных сценариях. Также были зафиксированы преимущества, недостатки и компромиссы, на которые пришлось пойти для улучшения метрик.

Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». B: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [3] Dolgov A. Kurochkin I. «Load balancing in grid systems from personal devices on the BOINC platform». B: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [4] Jerry Gao Mazedur Rahman Samira Iqbal. «Load Balancer as a Service in Cloud Computing». B: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [5] Lun WANG. «Low-Latency, High-Throughput Load Balancing Algorithms». B: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.